QA MASTER DOCUMENT

Databricks Data Export API

Contains:

• Test Plan • Test Analysis • Test Scenarios • Test Cases

Prepared by: Jeevan Paapagaari, Senior QA Engineer

SECTION 1 — TEST PLAN

1.1 Objective

This test plan validates the Databricks Data Export API, a production-ready, API-driven export capability built on Databricks Unity Catalog that enables customers to extract clinical trial data on scale. Testing covers three primary endpoints ***(POST /api/export, GET /api/export/status/{batch_id}, GET /api/export/tables)***, spanning asynchronous batch processing, multiple export modes (*Snapshot, Change Data Feed, Time-Travel*), OAuth2-based authentication and authorization for both enterprise and user types, file output formats (PARQUET and FEATHER), and delivery via Azure Blob Storage SAS URLs.

The plan ensures that all functional acceptance criteria, security requirements, performance expectations, and error-handling behaviors defined in the EPIC are traceable to executable test cases. Every Technical Task and Developer Instruction has corresponding test coverage, from initial authentication through end-to-end data retrieval.

1.2 EPIC Information

Field	Value
Epic ID	
Epic Title	**Databricks Data Export API**
Fix Version	
Team	Architecture Team
Work Category	New Feature / Platform Capability
QA Document Owner	Jeevan Paapagaari, Senior QA Engineer
Target Environment	Feature Branch → Dev Master → Stage5 → Production (Regional: NA, EU)

1.3 Scope

1.3.1 In Scope

API Endpoints Under Test:

Method	Path	Purpose
POST	*/api/export*	• Initiate asynchronous export batch • Returns batch_id immediately
GET	*/api/export/status/{batch_id}*	• Poll export status • Returns INPROGRESS / COMPLETED / FAILED / ERROR with download_url when done
GET	*/api/export/tables*	• Discover all exportable Databricks tables visible to the authenticated user
GET	*/health*	• Health check for API and critical dependencies (no auth required)

Functional Areas Covered:

- **OAuth2 Bearer Token Authentication**: Token validation, scope verification, token caching, custom grant_type dataexportapi
- **Enterprise User Authorization**: Customer ID extraction, API user validation, IP allow-listing, connection mapping
- **User Authorization**: User ID extraction, study access validation, IP allow-listing, study_id requirement
- **Batch Access Authorization:** Ownership tracking, per-user download access, 403 enforcements

- **Exportable Table Discovery:** Unity Catalog metadata, permission filtering, gold/silver schema, non-study lookup tables
- **Asynchronous Export Initiation**: Immediate batch_id return, background processing, concurrency
- **Snapshot Export Mode**: Full current state, default mode, no timestamp filters
- **Change Data Feed (CDF) Export:** Incremental changes since timestamp, all DML operations, metadata
- **Time-Travel Export (Not MVP):** Point-in-time historical data via Delta Lake, version/timestamp parameters
- **Parallel Query Execution**: Concurrent table export, configurable parallelism, fault isolation
- **File Format Handling**: PARQUET (ZIP_STORED) and FEATHER (ZIP_DEFLATED), extension validation, data type integrity
- **Azure Blob Storage:** File write, naming convention, retention, upload retry
- **SAS URL Generation:** Time-limited read-only tokens, download validation
- **Export Request Validation:** Field validation, table existence checks, mutual exclusivity of timestamp params
- **Error Handling:** Detailed error codes, partial failure handling, structured error responses
- **Retry Logic**: Exponential backoff, transient vs. permanent error distinction, configurable retry count
- **Structured Logging:** Request logging, lifecycle events, log levels, sensitive data exclusion
- **Health Check Endpoint:** Dependency checks, 200/503 status, unauthenticated access
- **Region Routing**: Instance_ID → Database_ID → Region_ID lookup chain, transparent redirection
- **Multi-Environment Configuration:** FeatureBranch/DevMaster/Stage5/Production env vars, startup validation, .env support

1.3.2 Out of Scope

- Front-end UI for export management (UI testing excluded; API-only scope)
- Infrastructure provisioning, Databricks cluster configuration, and Unity Catalog setup
- Identity Provider (IdP) internal implementation — only token validation behavior is in scope
- Azure Blob Storage account provisioning and access control configuration
- Non-Databricks API endpoints (other platform APIs unrelated to this EPIC)
- Performance load testing at production scale (covered under NFR scenarios; full load test suite is separate)
- Database migration or ETL pipeline validation outside the export API context
- CSV export format (noted in subtask documentation as a format option, but PARQUET/FEATHER are the production formats per Solution spec)

1.4 Test Environment Requirements

Component	Requirement	Details
Application	Databricks Export API service deployed	• Feature Branch, DevMaster, Stage5 Environments. • Each regional instance (NA, EU) accessible
Databricks	Unity Catalog with test schemas	• Silver and Gold schemas populated • CDF enabled on all test tables. • Delta Lake versioning active
Azure Blob Storage	Test storage account	• Configured per environment. • Write/read permissions • Retention policy set • SAS token generation enabled
Auth Provider	OAuth2 Identity Provider	• Token endpoint available. • Test credentials for Enterprise and users. • Custom grant_type dataexportapi registered
Test Tokens	Pre-issued OAuth2 bearer tokens	• Valid token (enterprise user)

Component	Requirement	Details
		• Valid token (user) • Expired token • Token missing scopes • Malformed token • Token without databricksexportapi scope
Test Datasets	Seeded clinical study data	• Min 2 studies with known study_ids • Tables with >10k rows for performance • Tables with version history for time-travel • Tables with recent changes for CDF
IP Allow-listing	Test IPs configured	• At least one authorized IP and • At least one unauthorized IP for validation testing
Environments (Optional)	Feature Branch, DevMaster and Stage5	• .env files per environment. • ENV variable switching mechanism documented
Monitoring	Logging infrastructure	• Log aggregation available. • Structured log output accessible for verification

1.5 Test Levels & Approach

1.5.1 API/System Testing (Primary)

All test cases are executed at the API layer using HTTP clients (e.g., Postman). Test cases validate request/response contracts, HTTP status codes, response body field correctness, async polling behavior, and header validation directly against deployed API endpoints.

1.5.2 Integration Testing

Integration test points requiring cross-system validation:

- **OAuth2 Identity Provider ↔ Export API:** Token issuance, scope validation, grant_type dataexportapi end-to-end
- **Export API ↔ Databricks Unity Catalog:** Table discovery, SQL query execution, CDF/time-travel queries
- **Export API ↔ Azure Blob Storage:** File write, naming convention, retention, SAS token generation and download
- **Region Routing:** Instance_ID/Database_ID/Region_ID lookup chain and request forwarding to regional API instances
- **Health Check ↔ All dependencies:** Database, Blob Storage, and Databricks connectivity verification

1.5.3 Regression Testing

Areas where regression must be validated after EPIC implementation:

- **Existing authentication flows:** Ensure OAuth2 changes do not break other API consumers
- **Study access permission model:** Validate existing study authorization rules are not relaxed
- **Platform API stability:** Confirm other APIs remain unaffected by new features_tbl/permission_tbl entries
- **Identity Server:** Verify new databricksexportapi validator does not interfere with existing token validation

1.6 Test Types

Test Type	What Will Be Tested	Test Cases In
Functional	• All happy-path scenarios for each endpoint and export mode • Parameter validation • Table filtering • File format output	Sections 3.1 – 3.7, 3.10 – 3.14

Test Type	What Will Be Tested	Test Cases In
	• Status lifecycle • Download via SAS URL • auth flows for both user types • batch ownership enforcement	
Security	• No token / expired token / malformed token / wrong scopes. • Unauthorized resource access. • Cross-user batch isolation. • IP allow-listing enforcement. • SAS URL read-only enforcement. • Sensitive data not in logs	Section 3.8
Non-Functional	• Response time for async initiation (<1s). • Parallel export throughput. • Health check response time (<1s). • Export completion time for large datasets. • SAS URL expiration timing	Section 3.9
Error Handling	• All defined HTTP error codes (400, 401, 403, 404, 500, 503) • Structured error messages; partial failure / table-level errors • Invalid parameter combinations. • Retry on transient failure. • Fail-fast on permanent failure	Sections 3.1, 3.2, 3.8, 3.13
Resilience	• Blob storage upload failure + retry. • Databricks connection failure. • Parallel export with one table failing. • Health check under dependency degradation. • Startup failure with missing env config	Sections 3.9, 3.13

1.7 Entry Criteria

- API contract (OpenAPI/Swagger specification) finalized and shared with QA
- Feature Branch environment deployed and accessible with all three endpoints responding
- OAuth2 Identity Provider configured with test users (Enterprise and) and dataexportapi grant_type
- Test bearer tokens issued (valid, expired, malformed, wrong-scope variants)
- Databricks Unity Catalog seeded with test data including silver/gold schemas and CDF-enabled tables
- Azure Blob Storage test account configured with write access for export API
- IP allow-list test entries configured (authorized and unauthorized IPs)
- Health check endpoint (/health) accessible without authentication
- Logging infrastructure is available to verify structured log output
- Region routing configuration documented (Instance_ID/Database_ID/Region_ID test data available)
- QA test plan reviewed and approved by Architecture and Development leads

1.8 Exit Criteria

- 100% of Critical priority test cases passed
- ≥95% of High priority test cases passed; any failures documented with accepted risk sign-off
- ≥90% of Medium priority test cases passed; all failures tracked as known issues
- All three API endpoints (*POST /api/export, GET /api/export/status/{batch_id}, GET /api/export/tables*) have at least one passing Critical happy-path test
- All security test cases executed; no open Critical or High security defects

- End-to-end test (TC-E2E-001) passes: authenticate → discover tables → initiate export → poll status → download file
- Regression test pass rate ≥95% for existing auth and study access flows
- No open P1/Blocker defects; all open P2 defects have accepted mitigation or deferred fix plan
- QA sign-off recorded in defect tracking system with final test execution results
- Test coverage summary (Section 4) completed and reviewed by stakeholders

1.9 Risks & Mitigation

Risk	Impact	Likelihood	Mitigation
OAuth2 Identity Provider not available in DEV for custom grant_type dataexportapi	High — Authentication tests (TC-SEC-001 to TC-SEC-005) blocked	Medium	• Coordinate with Identity Server team for early DEV configuration. • Use mock token service as fallback. • Prioritize auth integration in sprint planning
Databricks Unity Catalog CDF not enabled on test tables, preventing CDF and Time-Travel tests	High — All TC-CDF and TC-TTR test cases blocked	Medium	• Include table setup (CDF enable, version history seeding) in sprint zero. • Document setup runbook. • QA validates environment prior to test execution
Azure Blob Storage SAS URL expiration testing is time-sensitive and may be flaky in automation	Medium — TC-SDL-005, TC-FMT-005 may be intermittent	High	• Use configurable SAS expiry (set to very short duration, e.g., 30 seconds, in test env). • Add retry-with-wait logic in automation. • Manual verification as backup
Region routing requires multi-instance deployment (NA + EU) which may not be available in DEV	High — All TC-RGN scenarios blocked without dual-region setup	High	• Test routing logic at unit/integration level with mocked regional endpoints • Defer full E2E region routing to Feature Branch / DM • Document as known gap in DEV
Async export tests require polling and can have non-deterministic completion times, causing flaky tests	Medium — TC-SNP, TC-CDF, TC-STS scenarios may time out intermittently	High	• Implement configurable poll timeout and interval in test harness • Seed small datasets for testing • Define max acceptable wait time (e.g., 60 seconds for test exports)
Parallel execution testing (TC-NFR-001) may be resource-constrained in DEV environment	Medium — Cannot validate full parallelism benefit in low-resource env	Medium	• Use Feature Branch / DevMaster with production-like resources for performance validation. • Document DEV limitations; track as environment constraint does not defect

SECTION 2 — TEST ANALYSIS

2.1 Endpoints Analysis

Endpoint	Method	Primary Function	Risk Level
POST /api/export	POST	• Initiates async export batch. • Validates auth, params, tables. • Returns batch_id	Critical
GET /api/export/status/{batch_id}	GET	• Polls export status. • Returns INPROGRESS / COMPLETED / FAILED / ERROR. • Includes download_url on COMPLETED	Critical
GET /api/export/tables	GET	• Lists all exportable Databricks tables filtered by user permissions and schema	High
GET /health	GET	• Health check for API and all critical dependencies. • Unauthenticated. • <1s response	High

2.2 Parameter Analysis

2.2.1 POST /api/export — Request Body Parameters

Parameter	Type	Required?	Valid Values	Test Focus
study_ids	Array[UUID/GUID]	Optional (required for users)	• Valid UUID/GUIDs. • Max 20 entries. • Omit for enterprise auto-resolve	Empty array, >20 entries, invalid UUID/GUID format, unknown study_id, study not enabled for export
cdf_start_timestamp	String (datetime)	Optional	• ISO datetime • Must be before cdf_end_timestamp; • Mutually exclusive with as_of_timestamp	Future timestamp, timestamp before CDF enabled, combined with as_of_timestamp (invalid)
cdf_end_timestamp	String (datetime)	Optional	• ISO datetime • Must be after cdf_start_timestamp	Before start timestamp, equal to start timestamp, omitted (open-ended CDF)
as_of_timestamp	String (datetime)	Optional	• ISO datetime • Mutually exclusive with CDF timestamps	Combined with cdf_start_timestamp (invalid), timestamp beyond retention period, valid historical timestamp
table_names	Array[String]	Optional	• Valid Databricks table names • If omitted, all accessible tables exported	Empty array, non-existent table name, table user cannot access, mix of valid and invalid names
file_format	String (enum)	Optional	• PARQUET (default) or FEATHER	Omitted (default PARQUET), lowercase "parquet" (case sensitivity), invalid value "CSV", invalid value "JSON"

2.2.2 GET /api/export/status/{batch_id} — Path Parameters & Headers

Parameter	Type	Required?	Valid Values	Test Focus
batch_id (path)	UUID/GUID (string)	Required	• Valid UUID/GUID returned from POST /api/export	Non-existent batch_id, malformed UUID/GUID, batch_id belonging to different user, empty string
Authorization (header)	Bearer token	Required	• Valid OAuth2 bearer token	No header, expired token, malformed token, token without required scopes, token for different customer

2.2.3 GET /api/export/tables — Headers Only

Parameter	Type	Required?	Valid Values	Test Focus
Authorization (header)	Bearer token	Required	Valid OAuth2 bearer token	No token, expired token, enterprise vs user (different table sets expected)

2.3 Status/Response Lifecycle Analysis (Async API)

Status/State	Condition	Expected Fields in Response	download_url Present?	HTTP Code
INPROGRESS	• Export initiated. • Background processing underway	batch_id, status, message, timestamp	No	200
COMPLETED	• All tables exported successfully. • File zipped and stored in Azure Blob	batch_id, status, message, table_count, total_rows, timestamp, download_url	Yes (SAS URL)	200
FAILED	• Export failed before any table processed. • Non-recoverable	batch_id, status, message, timestamp	No	200
ERROR	• Partial failure. • Some tables failed. • Recovery not possible	batch_id, status, message, timestamp (table-level errors in message)	No (or partial)	200
Not Found	• Batch_id does not exist, or user not authorized	(error response)	No	404
Server Error	• Unexpected failure in status retrieval	(error response)	No	500

2.4 Equivalence Partitions & Boundary Values

Parameter	Valid Partition (Pass)	Invalid Partition (Fail)	Boundary Values
study_ids array	1–20 valid UUID/GUIDs	0 entries (user), >20 entries, non-UUID/GUID strings	Exactly 1 entry, exactly 20 entries, 21 entries
cdf_start_timestamp	Valid past datetime string, after CDF table creation	Future timestamp, timestamp before CDF enabled, combined with as_of_timestamp	Exact CDF table creation timestamp, 1ms before CDF enabled
as_of_timestamp	Valid historical timestamp within retention period	Timestamp beyond retention period, combined with CDF params, future timestamp	Exactly at retention boundary (30 days)

Parameter	Valid Partition (Pass)	Invalid Partition (Fail)	Boundary Values
table_names array	Existing tables user has access to	Non-existent tables, tables user cannot access, empty array	1 valid table, 1 valid + 1 invalid (mixed)
file_format	"PARQUET", "FEATHER"	Any other string ("CSV", "JSON", ""), null	"parquet" (lowercase — case sensitivity check)
batch_id path param	Valid UUID/GUID from prior POST /api/export	Non-existent UUID/GUID, malformed non-UUID/GUID string, empty string	Valid UUID/GUID from different user (cross-user isolation)
Authorization header	Valid bearer token with all required scopes	No header, expired token, malformed token, insufficient scopes	Token expiring in <5 seconds (race condition)

2.5 Security Analysis

- **Authentication Vectors:**
  - Missing Authorization header must return 401
  - Expired token must return 401
  - Malformed/invalid token must return 401
  - Token without required OAuth2 scopes (openid, bearer, offline_access, databricksexportapi) must return 401 with scope-related error
- **Authorization Rules:**
  - users must only access their own batches (403 for others).
  - Enterprise users can access all batches for their customers but no other customers (403)
  - study_id must be validated against user permissions before export is initiated
- **Cross-Resource Isolation**:
  - User A cannot poll status for User B's batch_id
  - Enterprise User A cannot access exports from Customer B
  - Cross-study access must be blocked for users without explicit study access
- **IP Allow-Listing:**
  - API calls from non-whitelisted IP addresses must be rejected.
  - IP validation must occur for both Enterprise and user types.
  - Allow-list enforcement must be logged
- **Attack Surfaces:**
  - Injection via study_ids (UUID/GUIDs should be validated, not used raw in queries)
  - Path traversal via batch_id
  - Oversized array payloads (study_ids >20 must be rejected.
  - table_names should have a reasonable limit)
- **SAS URL Security:**
  - Generated SAS URLs must be read-only (cannot delete or write to blob)
  - URLs must expire after configured duration
  - Expired SAS URLs must return 403/404 from Azure Blob Storage
  - SAS URLs must not be guessable
- **Data Leakage Risks:**
  - Sensitive data (OAuth tokens, PII from clinical records) must not appear in log outputs
  - Error messages must not expose internal implementation details (stack traces, DB queries)
  - batch_id should not be predictable/sequential
- **Token Caching Security:**
  - Cached tokens must be invalidated on expiry
  - Negative cache (invalid tokens) should have short TTL to prevent lockout
  - Cache must not serve one user's token to another

2.6 Non-Functional Analysis

NFR Area	Scenario	Expected Behaviour
Performance: Initiation	POST /api/export called for any valid request	• Response with batch_id returned in <500ms • Processing happens asynchronously
Performance: Health Check	GET /health called under normal conditions	• Response returned in <1 second including all dependency checks
Performance: Table Discovery	GET /api/export/tables called for user with many accessible tables	• Response returned in <2 seconds. • JSON pagination if >1000 tables
Scalability: Parallel Export	5 tables exported concurrently in single batch	• All tables processed in parallel. • Total time < sum of individual times. • No resource exhaustion
Scalability: Concurrent Batches	10 simultaneous export batches from different users	• All batches were initiated successfully. • No cross-batch interference. • Queue management applies if at capacity
Scalability: Large Dataset	Single table export with 1M+ rows	• Export completes without time out or memory error. • Status reflects INPROGRESS then COMPLETED. • File accessible via SAS URL
Resilience: Partial Failure	One table in a multi-table export fails during processing	• Other tables continue processing. • Final status reflects partial failure with table-level error details. • Error logged
Resilience: Blob Upload Failure	Azure Blob Storage upload fails transiently	• Retry logic activates. • Exponential backoff applied. • Success on subsequent attempt. • Retry count logged
Resilience: Startup Config Missing	API started with required environment variable absent	• Startup fails immediately with clear error message identifying missing config. • No partial initialization
Rate Limiting	Excessive API calls from a single client	• Rate limiting response (429 Too Many Requests) with retry-after header if rate limiting is implemented. • Graceful degradation

2.7 Region Routing Analysis

When a POST /api/export request is received, the system must perform a 3-step lookup chain to identify the correct regional instance and forward the request transparently to the user:

1. **Step 1:** Resolve Instance_ID: Look up study from study_ids in request to get Instance_ID
2. **Step 2:** Resolve Database_ID: Use Instance_ID to get Database_ID from Instance table
3. **Step 3:** Resolve Region_ID: Use Database_ID to get Region_ID from Databases table
4. **Step 4:** Forward Request: Route to appropriate regional API instance (e.g., EU_.DatabricksExport.API) transparently

Routing Step	Input	Output	Failure Mode	Expected Behaviour on Failure
1. Study → Instance	study_id from request	Instance_ID	study_id not found in registry	Return 404 with clear error: study not found

Routing Step	Input	Output	Failure Mode	Expected Behaviour on Failure
2. Instance → Database	Instance_ID	Database_ID	Instance_ID not mapped to database	Return 500 with logged internal error
3. Database → Region	Database_ID	Region_ID	Database_ID not mapped to region	Return 500 with logged internal error
4. Region → API Forward	Region_ID + original request	Response from regional API	Regional instance unreachable	Return 503 with retry guidance; log routing failure

2.8 Export Mode Mutual Exclusivity Analysis

Export Mode	Parameters Used	Mutually Exclusive With	Default?	Test Focus
Snapshot	None (no timestamps)	N/A	Yes (default when no timestamps supplied)	• Confirm no timestamp filtering • All active records returned
Change Data Feed (CDF)	cdf_start_timestamp (required), cdf_end_timestamp (optional)	as_of_timestamp	No	• Confirm only changes after start returned • All DML ops • Open-ended when end omitted
Time-Travel	as_of_timestamp	cdf_start_timestamp, cdf_end_timestamp	No	• Confirm exact historical state at timestamp • Error on out-of-retention requests
INVALID — Mixed	cdf_start_timestamp + as_of_timestamp supplied together	Each other	N/A	• Must return 400 Bad Request. • Clear mutual exclusive error message

2.9 Retry Logic Analysis

Failure Type	Classification	Retry Behaviour	Max Retries	Post-Retry State
Azure Blob Storage transient network error	Transient	Exponential backoff and retry automatically	Configurable (default TBD)	Success OR permanent failure with ERROR status after exhaustion
Databricks SQL warehouse connection timeout	Transient	Exponential backoff and retry automatically	Configurable	Success OR ERROR status with table-level error detail
Invalid OAuth2 token	Permanent	No retry	N/A	401 returned immediately
Table not found in Unity Catalog	Permanent	No retry	N/A	404 / validation error returned; export not queued
Databricks warehouse unavailable (503)	Transient	Exponential backoff	Configurable	Success OR FAILED status with reason
Application server OOM / crash	Permanent (from retry perspective)	No retry (new request required)	N/A	FAILED status; logged; health check reflects unhealthy

SECTION 3 — TEST SCENARIOS & TEST CASES

3.1 POST /api/export — Initiate Asynchronous Export

This scenario validates the export initiation endpoint. It covers the happy-path for both Enterprise and users, all request parameter combinations (snapshot, CDF, time-travel, file formats, table filtering), validation of mutual exclusivity rules, and error responses for invalid inputs. The endpoint must return a batch_id immediately (async) with HTTP 202 for all valid requests.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-EXP-001	**Successful snapshot export — Enterprise user, default PARQUET (Sample StudyIds taken)**	1. Enterprise user with valid bearer token authenticated 2. study_ids are valid and belong to customers 3. Databricks tables with study_id column exist	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "file_format": "PARQUET"} 2. Include Authorization: Bearer <valid-enterprise-token> 3. Record response	1. HTTP 202 Accepted returned 2. Response body contains "batch_id" (valid UUID/GUID) 3. Response body contains "tables_queued" (integer > 0) 4. Response returned in < 500ms 5. No export file available yet (processing is async)	**Critical**	Functional
TC-EXP-002	**Successful snapshot export — user with explicit study_ids**	1. user with valid bearer token authenticated 2. User has access to the specified studies 3. Databricks tables exist for those studies	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000", "6db0f65f-17e9-4d6d-983f-d4380a26e217"], "file_format": "PARQUET"} 2. Include Authorization: Bearer <valid--token> 3. Record batch_id from response	1. HTTP 202 Accepted 2. "batch_id" present (UUID/GUID) 3. "tables_queued" >= number of tables for those studies 4. Export begins processing in background	**Critical**	Functional
TC-EXP-003	**Export with table_names filter — only specified tables included**	1. Authenticated user with valid token 2. "patients" and "visits" tables exist and are accessible	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "table_names": ["patients", "visits"]} 2. Include Authorization: Bearer <valid-token> 3. Poll status until COMPLETED 4. Download and inspect ZIP file contents	1. HTTP 202 Accepted 2. tables_queued = 2 3. On completion, ZIP contains exactly "patients" and "visits" files 4. No other table files present in ZIP	**High**	Functional
TC-EXP-004	**Export without table_names — all accessible tables exported**	1. Authenticated user 2. Multiple accessible tables exist for the study	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"]} (no table_names)	1. HTTP 202 Accepted 2. tables_queued equals total number of	**High**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
			2. Include Authorization: Bearer <valid-token>	accessible tables for the study 3. All tables appear in completed export		
TC-EXP-005	**CDF export with start and end timestamps**	1. Authenticated user 2. CDF enabled on test tables 3. Known changes made between cdf_start_timestamp and cdf_end_timestamp	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "cdf_start_timestamp": "2025-09-30 17:10:45", "cdf_end_timestamp": "2025-09-30 18:00:00"} 2. Include Authorization: Bearer <valid-token>	1. HTTP 202 Accepted 2. batch_id returned 3. On completion, exported data contains only changes in the specified window 4. Change metadata (operation type, timestamp) included in records	**Critical**	Functional
TC-EXP-006	**Time-travel export with as_of_timestamp**	1. Authenticated user 2. Delta Lake active versioning on test tables 3. Known historical state at target timestamp	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "as_of_timestamp": "2025-09-30 17:10:45"} 2. Include Authorization: Bearer <valid-token>	1. HTTP 202 Accepted 2. Exported data reflects table state at specified timestamp 3. Rows added after timestamp not included 4. Rows deleted before timestamp not included	**High**	Functional
TC-EXP-007	**FEATHER format export**	1. Authenticated user 2. Tables accessible	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "file_format": "FEATHER"} 2. Include Authorization: Bearer <valid-token> 3. On completion, download ZIP	1. HTTP 202 Accepted 2. Exported files have .feather extension 3. ZIP created with ZIP_DEFLATED compression 4. Files are valid Apache Feather format (readable by pandas/R)	**High**	Functional
TC-EXP-008	**INVALID — cdf_start_timestamp and as_of_timestamp both provided**	1. Authenticated user	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "cdf_start_timestamp": "2025-09-30 17:10:45", "as_of_timestamp": "2025-09-30 17:10:45"} 2. Include Authorization: Bearer <valid-token>	1. HTTP 400 Bad Request 2. Response body contains error message indicating mutual exclusivity violation	**Critical**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
				3. No batch_id returned 4. Export is NOT queued		
TC-EXP-009	**INVALID — study_ids array exceeds 20 entries**	1. Authenticated user	1. Construct body with "study_ids" array of 21 valid UUID/GUIDs 2. POST /api/export 3. Include Authorization: Bearer <valid-token>	1. HTTP 400 Bad Request 2. Error message indicates maximum of 20 study_ids exceeded 3. No batch_id returned	**High**	Functional
TC-EXP-010	**INVALID — non-existent table name in table_names**	1. Authenticated user	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "table_names": ["nonexistent_table_xyz"]} 2. Include Authorization: Bearer <valid-token>	1. HTTP 404 Not Found 2. Response indicates no matching tables found 3. Export NOT queued	**High**	Functional
TC-EXP-011	**user omits study_ids — validation error**	1. user with valid bearer token	1. POST /api/export with body: {"file_format": "PARQUET"} (no study_ids) 2. Include Authorization: Bearer <valid--token>	1. HTTP 400 Bad Request 2. Error message indicates study_id is required for users 3. No batch_id returned	**Critical**	Functional
TC-EXP-012	**Invalid file_format value**	1. Authenticated user	1. POST /api/export with body: {"study_ids": ["550e8400-e29b-41d4-a716-446655440000"], "file_format": "CSV"} 2. Include Authorization: Bearer <valid-token>	1. HTTP 400 Bad Request 2. Error message indicates invalid file_format; valid options are PARQUET and FEATHER 3. No batch_id returned	**High**	Functional

3.2 GET /api/export/status/{batch_id} — Export Status & Download

This scenario validates the status polling endpoint across all lifecycle states (INPROGRESS, COMPLETED, FAILED, ERROR), verifies the presence and correctness of response fields at each state, confirms download_url is returned only upon COMPLETED status, and validates error handling for invalid or unauthorized batch_id values.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-STS-001	**Poll status immediately after export initiation — INPROGRESS**	1. Export initiated via TC-EXP-001; batch_id recorded 2. Export has not completed yet	1. GET /api/export/status/{batch_id} with valid batch_id 2. Include Authorization: Bearer <valid-token> 3. Send request within 2 seconds of initiation	1. HTTP 200 OK 2. Response contains: batch_id (matches request), status ("INPROGRESS"), message (non-empty	**Critical**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
				string), timestamp (ISO format) 3. download_url field is NOT present 4. table_count and total_rows are NOT present		
TC-STS-002	**Poll status after export completes — COMPLETED with download_url**	1. Export initiated and sufficient time elapsed for completion 2. Tables contain small dataset for fast completion	1. POST /api/export to initiate 2. Poll GET /api/export/status/{batch_id} every 5 seconds 3. Continue until status = "COMPLETED" (max 60 seconds) 4. Record full response	1. HTTP 200 OK 2. status = "COMPLETED" 3. message = "Export completed successfully" (or equivalent success message) 4. table_count = number of tables exported (integer) 5. total_rows = total rows across all tables (integer) 6. timestamp present (ISO format) 7. download_url present and starts with "https://" 8. download_url is a valid Azure SAS URL (contains "blob.core.windows.net")	**Critical**	Functional
TC-STS-003	**Download file via SAS URL from COMPLETED export**	1. Export COMPLETED (TC-STS-002 passed) 2. download_url recorded	1. Send HTTP GET request to the download_url from the COMPLETED status response 2. No Authorization header needed (SAS URL is self-authenticating) 3. Save response body as ZIP file 4. Extract ZIP and inspect contents	1. HTTP 200 returned from Azure Blob Storage (or 302 redirect then 200) 2. Content-Type indicates ZIP/octet-stream 3. ZIP contents include one file per exported table 4. Files have correct extension (.parquet or .feather per file_format requested) 5. Files are valid and readable in appropriate tool	**Critical**	Functional
TC-STS-004	**SAS URL expires after configured duration**	1. Export COMPLETED; download_url recorded 2. Test environment configured with short SAS expiry (e.g., 30 seconds)	1. Record download_url from COMPLETED status 2. Wait for expiry duration + 5 seconds 3. Attempt HTTP GET to the download_url	1. Request to expired SAS URL returns HTTP 403 or 404 from Azure Blob Storage 2. Response contains error indicating token expired or invalid 3. Original export status endpoint still returns COMPLETED (status persisted)	**High**	Functional
TC-STS-005	**Status for non-existent batch_id — 404**	1. Authenticated user with valid token	1. GET /api/export/status/00000000-0000-0000-0000-000000000000 2. Include Authorization: Bearer <valid-token>	1. HTTP 404 Not Found 2. Response body contains error message "Batch not found" (or equivalent) 3. No download_url in response	**High**	Functional
TC-STS-006	**Status for malformed**	1. Authenticated user with valid token	1. GET /api/export/status/not-a-valid-UUID/GUID	1. HTTP 400 Bad Request or 404 Not Found	**Medium**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
	batch_id (not a UUID/GUID)		2. Include Authorization: Bearer <valid-token>	2. Response contains appropriate error message		
TC-STS-007	**FAILED status response after export failure**	1. Export initiated for a study that will fail (e.g., Databricks warehouse offline) 2. Retry logic exhausted	1. POST /api/export to initiate export against unavailable resource 2. Poll GET /api/export/status/{batch_id} until final state 3. Record response when status = "FAILED"	1. HTTP 200 OK 2. status = "FAILED" 3. message contains descriptive failure reason 4. download_url is NOT present 5. Failure is logged with full context	**High**	Functional
TC-STS-008	**Detailed status shows table-level progress**	1. 1. Export initiated for multiple tables 2. Export in progress	1. POST /api/export for 3 tables 2. Poll status every 3 seconds while INPROGRESS 3. Inspect response body at each poll	1. Status response includes table-level progress details (per the Detailed Status subtask) 2. Record counts shown per table during/after processing 3. Error messages present at table level if any table fails	**High**	Functional

3.3 GET /api/export/tables — Exportable Table Discovery

This scenario validates the table discovery endpoint. It ensures the correct tables are returned based on user type and permissions, that gold and silver schema tables are properly distinguished, that response format is valid JSON, and that table metadata (schema, description) is included.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-TBL-001	**List exportable tables — authenticated enterprise user**	1. Enterprise user with valid bearer token 2. Multiple tables exist in Databricks Unity Catalog	1. GET /api/export/tables 2. Include Authorization: Bearer <valid-enterprise-token> 3. Parse response JSON	1. HTTP 200 OK 2. Response is valid JSON 3. Response contains "tables" array and "count" integer 4. Each table object contains: table_name (string), table_type (e.g., "MANAGED"), columns array 5. Each column object contains "name" and "type_text" 6. Count matches length of tables array	**Critical**	Functional
TC-TBL-002	**Table names match actual Databricks table names**	1. Enterprise user authenticated 2. Known set of tables exists in Databricks	1. GET /api/export/tables with valid token 2. Compare returned table_names against known Databricks table list	1. All expected tables present in response 2. No phantom tables returned (tables that don't exist in Databricks) 3. table_name values exactly match Databricks Unity Catalog names (case-sensitive)	**Critical**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-TBL-003	**Tables filtered by user permissions — vs Enterprise**	1. user with access to Study A 2. Enterprise user with access to all studies	1. GET /api/export/tables with user token; record results 2. GET /api/export/tables with Enterprise user token; record results 3. Compare results	1. user sees only tables for their authorized studies 2. Enterprise user sees all tables for their customer 3. user does NOT see tables for studies they have no access to 4. Result sets differ as expected	**Critical**	Functional
TC-TBL-004	**Gold schema tables included and identified in response**	1. Authenticated enterprise user 2. Gold schema tables exist in Databricks	1. GET /api/export/tables with valid token 2. Filter results for gold schema tables	1. Gold schema tables appear in response alongside silver tables 2. Response metadata clearly indicates table schema type (gold vs silver) 3. Gold tables filterable by schema identifier	**High**	Functional
TC-TBL-005	**Non-study lookup tables included if configured**	1. Configurable non-study tables list populated 2. Authenticated user	1. GET /api/export/tables with valid token 2. Look for tables without study_id column in response	1. Configured lookup tables (without study_id column) appear in table list 2. These tables are exportable 3. Tables are clearly identifiable as non-study tables if metadata supports it	**Medium**	Functional
TC-TBL-006	**No authorization — 401 response**	1. No token available	1. GET /api/export/tables without Authorization header	1. HTTP 401 Unauthorized 2. Response indicates authentication required 3. No table data returned	**Critical**	Security

3.4 Snapshot Export Mode

This scenario validates the snapshot export mode specifically — the default behavior that returns the complete current state of all requested tables with no timestamp filtering. Tests confirm that all active records are present, that deleted records are excluded, and that snapshot is correctly applied as the default mode when no export mode parameters are supplied.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-SNP-001	**Snapshot returns all current active records**	1. Authenticated user 2. Test table "patients" contains known set of 50 active records 3. No timestamps in request body	1. POST /api/export with body: {"study_ids": ["<study_id>"], "table_names": ["patients"]} 2. Poll until COMPLETED 3. Download and parse export file 4. Count records	1. Export completes successfully 2. Exported file contains exactly 50 records 3. All currently active patient records present 4. No timestamp metadata columns added (not a CDF export)	**Critical**	Functional
TC-SNP-002	**Snapshot is the default mode when no**	1. Authenticated user 2. Tables with historical changes exist	1. POST /api/export with only study_ids and file_format (no cdf or as_of timestamps) 2. Poll until COMPLETED	1. Export mode defaults to Snapshot 2. Data matches current table state exactly	**Critical**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
	timestamps supplied		3. Compare result with known current table state	3. No delta or historical filtering applied 4. Export includes all active records		
TC-SNP-003	**Snapshot excludes soft-deleted records appropriately**	1. Test table with soft-deleted rows (deleted flag set) 2. Authenticated user	1. POST /api/export snapshot for table with known deleted records 2. Download and inspect results	1. Soft-deleted records are NOT included if Databricks view excludes them 2. Active records all present 3. Row count matches expected active record count	**High**	Functional
TC-SNP-004	**Snapshot of large table completes without timeout**	1. Authenticated user 2. Large test table with >100,000 rows available	1. POST /api/export for large table with Snapshot mode 2. Poll until COMPLETED (max 5 minutes for large dataset) 3. Verify row count	1. Export completes without error (status = COMPLETED) 2. total_rows reflects expected row count 3. Downloaded file is not corrupted 4. File readable and data intact	**High**	Non-Functional
TC-SNP-005	**Multiple tables in one snapshot batch**	1. Authenticated user 2. Three tables accessible: patients, visits, adverse_events	1. POST /api/export with table_names: ["patients", "visits", "adverse_events"] 2. Poll until COMPLETED 3. Extract ZIP; inspect file list	1. tables_queued = 3 in 202 response 2. COMPLETED status shows table_count = 3 3. ZIP contains exactly 3 files with correct names and extensions 4. Each file is independently valid	**High**	Functional

3.5 Change Data Feed (CDF) Export Mode

This scenario validates the Change Data Feed export mode, which returns only row-level changes (INSERT, UPDATE, DELETE) between specified timestamps using Delta Lake CDF capabilities. Tests verify that only changes within the window are returned, that all DML operation types are captured, that change metadata is correct, and that CDF is more efficient than full snapshots for incremental use cases.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-CDF-001	**CDF export returns only changes since start timestamp**	1. CDF enabled on test table "patients" 2. Known inserts/updates performed after cdf_start_timestamp 3. Authenticated user	1. POST /api/export with: {"study_ids": ["<study_id>"], "cdf_start_timestamp": "2025-09-30 17:10:45", "table_names": ["patients"]} 2. Poll until COMPLETED 3. Download and parse exported file 4. Compare records against known changes made after timestamp	1. Export contains only records changed after cdf_start_timestamp 2. Records unchanged before that timestamp are NOT included 3. Record count matches expected number of changes	**Critical**	Functional
TC-CDF-002	**CDF export captures INSERT operations**	1. Known INSERT performed on test table at known timestamp	1. POST /api/export CDF mode with cdf_start_timestamp before the INSERT 2. Poll until COMPLETED	1. Inserted row appears in export 2. Change metadata includes:	**Critical**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
	with metadata	2. CDF enabled	3. Parse records and filter by operation type	_change_type = "insert", _commit_timestamp (correct timestamp) 3. Row data matches inserted record		
TC-CDF-003	**CDF export captures UPDATE operations**	1. Known UPDATE performed on test record 2. CDF enabled	1. POST /api/export CDF mode 2. Download and check update records	1. Updated row appears with _change_type = "update_postimage" (or equivalent) 2. Pre-update image present as "update_preimage" if API supports before/after values 3. Updated field values reflect the change	**Critical**	Functional
TC-CDF-004	**CDF export captures DELETE operations**	1. Known DELETE performed on test record 2. CDF enabled	1. POST /api/export CDF mode 2. Download and check delete records	1. Deleted row appears with _change_type = "delete" 2. Row data reflects deleted record's last values 3. _commit_timestamp present	**Critical**	Functional
TC-CDF-005	**CDF with start and end timestamp window**	1. Changes known in window [T1, T2] 2. Changes also exist before T1 and after T2 3. CDF enabled	1. POST /api/export with cdf_start_timestamp = T1 and cdf_end_timestamp = T2 2. Poll until COMPLETED 3. Inspect all record timestamps	1. Only records changed in [T1, T2] window returned 2. Records with commit_timestamp < T1 NOT present 3. Records with commit_timestamp > T2 NOT present	**High**	Functional
TC-CDF-006	**CDF without end timestamp — open-ended export to current**	1. Known changes after cdf_start_timestamp 2. CDF enabled	1. POST /api/export with only cdf_start_timestamp (no cdf_end_timestamp) 2. Poll until COMPLETED	1. All changes from cdf_start_timestamp to the time of export initiation are included 2. No changes are missed 3. Export completes successfully	**High**	Functional
TC-CDF-007	**CDF export is more efficient than snapshot for incremental use**	1. Large table with majority of records unchanged; small set of recent changes 2. CDF enabled	1. Measure duration and file size of snapshot export 2. Measure duration and file size of CDF export for same period with few changes 3. Compare results	1. CDF export file size significantly smaller than snapshot 2. CDF export duration is significantly shorter than snapshot 3. CDF record count matches expected changed records (subset of total)	**Medium**	Non-Functional

3.6 Time-Travel Export Mode (Not MVP)

This scenario validates the time-travel export mode, which leverages Delta Lake TIMESTAMP AS OF to return a point-in-time snapshot of table data. It is marked Not MVP but must be validated. Tests confirm correct historical data retrieval, version number support, and appropriate error responses for out-of-retention requests.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-TTR-001	**Time-travel returns data state at specific past timestamp**	1. Delta Lake versioning active on test table 2. Known historical state at target timestamp documented 3. Authenticated user	1. POST /api/export with body: {"study_ids": ["<study_id>"], "as_of_timestamp": "2025-09-30 17:10:45", "table_names": ["patients"]} 2. Poll until COMPLETED 3. Download and compare against known historical state at that timestamp	1. Export contains table state as it existed at 2025-09-30 17:10:45 2. Rows added after that timestamp are NOT present 3. Rows deleted before that timestamp are NOT present 4. Row values reflect state at that point in time	**High**	Functional
TC-TTR-002	**Time-travel beyond retention period returns clear error**	1. Authenticated user 2. Retention period known (e.g., 30 days)	1. POST /api/export with as_of_timestamp set to 31+ days ago 2. Include valid Authorization token	1. HTTP 400 Bad Request or 404 Not Found 2. Response message clearly indicates data not retained for the requested time 3. Export is NOT initiated	**High**	Functional
TC-TTR-003	**Time-travel and CDF timestamps mutually exclusive — 400 error**	1. Authenticated user	1. POST /api/export with body: {"study_ids": ["<study_id>"], "as_of_timestamp": "2025-09-30 17:10:45", "cdf_start_timestamp": "2025-09-30 16:00:00"} 2. Include valid Authorization token	1. HTTP 400 Bad Request 2. Error message states as_of_timestamp and CDF timestamps are mutually exclusive 3. Export NOT queued	**Critical**	Functional
TC-TTR-004	**Time-travel with future timestamp returns error**	1. Authenticated user	1. POST /api/export with as_of_timestamp set to tomorrow's date	1. HTTP 400 Bad Request 2. Error message indicates future timestamps are not valid for time-travel exports	**Medium**	Functional
TC-TTR-005	**Time-travel accuracy confirmed by cross-referencing known version**	1. Test table with multiple known versions at specific timestamps 2. 2. Version history documented	1. Request time-travel export at version V1 timestamp 2. Request time-travel export at version V2 timestamp 3. Compare results against known states	1. V1 export exactly matches known V1 state 2. V2 export exactly matches known V2 state 3. No data from V2 appears in V1 export and vice versa	**High**	Functional

3.7 File Storage — Azure Blob & SAS URL Delivery

This scenario validates Azure Blob Storage file writing, naming conventions, and SAS URL generation. It ensures files are correctly organized by batch_id and table name, that blob names follow the expected pattern, that SAS URLs are read-only and time-limited, and that download works from any location without IP restrictions.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-SDL-001	**Export files written to Azure Blob Storage with correct naming**	1. Export COMPLETED 2. Azure Blob Storage accessible for inspection	1. Initiate and complete an export for 2 tables 2. Access Azure Blob Storage container 3. List files under the batch_id prefix	1. Files exist under path: {batch_id}/{table_name}_{timestamp}.{format} 2. File names include correct batch_id, table name, timestamp, and extension 3. Files organized by batch_id prefix 4. Count of files equals number of tables exported	**High**	Functional
TC-SDL-002	**SAS URL provides read-only access (cannot write or delete)**	1. Export COMPLETED with download_url	1. Attempt HTTP GET to download_url — verify success 2. Attempt HTTP PUT to same URL with test content 3. Attempt HTTP DELETE to same URL	1. GET request succeeds (200 or redirect) 2. PUT request fails with 403 Forbidden or 405 Method Not Allowed 3. DELETE request fails with 403 Forbidden 4. SAS token scopes confirmed read-only	**Critical**	Security
TC-SDL-003	**SAS URL works from different IP addresses**	1. Export COMPLETED with download_url 2. Two different network locations available	1. Download file via download_url from Location A 2. Download file via download_url from Location B (different IP)	1. Both downloads succeed 2. File content is identical from both locations 3. No IP-based restriction on SAS URL access	**High**	Functional
TC-SDL-004	**Files retained for configured retention period**	1. Export COMPLETED 2. Retention period configured and known (e.g., 7 days)	1. Download export immediately after completion 2. Wait until just before retention expiry; download again 3. Wait until after retention expiry; attempt download	1. Download succeeds immediately after completion 2. Download succeeds just before retention expiry 3. Download fails after retention period (file deleted or SAS expired)	**Medium**	Functional
TC-SDL-005	**Blob upload failure triggers retry and succeeds**	1. Test environment with network fault injection capability 2. Authenticated user	1. Configure transient failure for first blob upload attempt 2. Initiate export 3. Monitor logs for retry attempts 4. Verify outcome	1. First upload attempt fails (transient error) 2. Retry logic activates with exponential backoff 3. Subsequent retry succeeds 4. Export ultimately reaches COMPLETED status 5. Retry attempts logged with timestamps and outcomes	**High**	Resilience
TC-SDL-006	**One URL per exported table in COMPLETED response**	1. Export with 3 tables COMPLETED	1. GET /api/export/status/{batch_id} after COMPLETED 2. Parse download_url — it should be a single ZIP URL	1. Single download_url present pointing to ZIP archive 2. ZIP contains one file per table 3. File names within ZIP are clearly identifiable by table name	**High**	Functional

3.8 Security — Authentication, Authorization & Data Isolation

This scenario provides comprehensive security coverage for all three endpoints. It validates every authentication failure mode (missing, expired, malformed, wrong-scope tokens), authorization enforcement (user type, study access, batch ownership), cross-user isolation, IP allow-listing, and confirms that sensitive data does not appear in logs.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-SEC-001	**No Authorization header — 401 on all endpoints**	1. No token available	1. POST /api/export without Authorization header 2. GET /api/export/status/<any-id> without Authorization header 3. GET /api/export/tables without Authorization header	1. All three requests return HTTP 401 Unauthorized 2. Response body indicates authentication required 3. No data or batch_id returned in any case	**Critical**	Security
TC-SEC-002	**Expired token — 401**	1. Expired bearer token available	1. POST /api/export with Authorization: Bearer <expired-token> 2. GET /api/export/tables with expired token	1. Both requests return HTTP 401 Unauthorized 2. Error message indicates token expired 3. Expired token not cached as valid; re-request returns same 401	**Critical**	Security
TC-SEC-003	**Malformed token — 401**	1. Malformed token string prepared	1. POST /api/export with Authorization: Bearer not.a.real.token 2. GET /api/export/status/<id> with malformed token	1. Both requests return HTTP 401 Unauthorized 2. Error indicates invalid token format	**Critical**	Security
TC-SEC-004	**Token without required scopes — 401**	1. Valid OAuth2 token but missing "databricksexportapi " scope	1. POST /api/export with Authorization: Bearer <token-without-databricksexportapi-scope> 2. Include otherwise valid request body	1. HTTP 401 Unauthorized 2. Error message indicates required scope missing 3. Export NOT initiated	**Critical**	Security
TC-SEC-005	**Custom grant_type dataexportapi token accepted end-to-end**	1. Token issued with grant_type=dataexportapi 2. All required scopes present	1. Obtain token using grant_type=dataexportapi 2. POST /api/export with this token 3. GET /api/export/tables with this token	1. Both requests succeed (202 and 200 respectively) 2. Token validated successfully 3. No auth errors	**Critical**	Security
TC-SEC-006	**user cannot access another user's batch — 403**	1. User A () has created export batch (batch_id_A) 2. User B () is authenticated	1. Authenticate as User B 2. GET /api/export/status/{batch_id_A} with User B's token	1. HTTP 403 Forbidden 2. Error message indicates unauthorized access to this batch 3. No status information revealed to User B 4. Unauthorized access attempt is logged	**Critical**	Security
TC-SEC-007	**Enterprise user cannot access export from different customer**	1. Customer A has exports 2. Enterprise User B is from Customer B	1. Authenticate as Enterprise User B 2. GET /api/export/status/{batch_id_from_customer_A}	1. HTTP 403 Forbidden 2. No export data from Customer A returned to Customer B	**Critical**	Security

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-SEC-008	**user cannot export study they do not have access to**	1. user with access to Study A, NOT Study B 2. Both study IDs known	1. POST /api/export with study_ids: [study_B_id] 2. Use user's token	1. HTTP 403 Forbidden or 404 Not Found 2. No batch_id returned 3. Authorization failure logged	**Critical**	Security
TC-SEC-009	**IP allow-listing — request from unauthorized IP blocked**	1. Known unauthorized IP available (not in allow-list) 2. Test token valid	1. Send POST /api/export from unauthorized IP address with valid token	1. HTTP 403 Forbidden (or 401, depending on implementation order) 2. Error message indicates IP not authorized 3. IP validation failure logged	**High**	Security
TC-SEC-010	**IP allow-listing — authorized IP passes**	1. Test from IP in the allow-list 2. Valid token	1. Send POST /api/export from authorized IP with valid token	1. Request accepted normally (HTTP 202) 2. batch_id returned	**High**	Security
TC-SEC-011	**Study not enabled for Databricks Export API — access denied**	1. Study exists but "Databricks Export API" feature NOT enabled for it 2. User has study access	1. POST /api/export with study_id of non-enabled study 2. Use valid user token	1. HTTP 403 Forbidden 2. Error indicates study is not enabled for data export 3. Export NOT initiated	**High**	Security
TC-SEC-012	**Sensitive data (tokens, PII) not present in logs**	1. Logging infrastructure accessible 2. Export request with valid token made	1. Make a successful POST /api/export request 2. Access application logs for the request 3. Search for token value in logs 4. Search for any PII fields from export data in logs	1. Bearer token value does NOT appear in any log entry 2. Patient PII from export content does NOT appear in logs 3. Logs contain request metadata (method, path, user_id, response_code) but NOT token or data values	**High**	Security

3.9 Non-Functional & Resilience Testing

This scenario covers performance, scalability, and resilience requirements. Tests validate response time SLAs, concurrent export handling, parallel execution behavior, health check performance, and system recovery from various failure modes including dependency outages and configuration errors.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-NFR-001	**POST /api/export returns batch_id in < 500ms**	1. API deployed in STAGING 2. Valid export request ready	1. Record start time (t0) 2. POST /api/export with valid request body and token 3. Record time when 202 response received (t1) 4. Calculate t1 - t0	1. HTTP 202 received in < 500ms 2. batch_id present in response 3. Result consistently < 500ms across 5 repeated calls	**High**	Non-Functional
TC-NFR-002	**Health check responds in < 1 second**	1. All dependencies healthy 2. /health endpoint deployed	1. GET /health (no auth required) 2. Measure response time across 5 consecutive calls	1. HTTP 200 OK each time 2. Response time < 1000ms for each call	**High**	Non-Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
				3. Response includes status for each critical dependency 4. All dependencies show healthy		
TC-NFR-003	**Parallel export of multiple tables — concurrent processing confirmed**	1. STAGING environment with Databricks SQL warehouse 2. Export batch with 5 tables	1. Initiate export for 5 tables 2. Monitor processing; record per-table start/end times via logs 3. Compare total time vs sum of individual table times	1. Multiple tables shown as processing simultaneously in logs 2. Total export time < sum of sequential table times (parallel speedup) 3. All 5 tables complete successfully 4. No resource errors or throttling	**High**	Non-Functional
TC-NFR-004	**One failing table does not block other tables in batch**	1. Export batch with 3 tables 2. One table configured to fail (removed from Databricks or privileges revoked)	1. Initiate export for 3 tables (1 set to fail) 2. Poll status until final state 3. Download results if available	1. Two successful tables complete and are available for download 2. One table shows table-level error in status response 3. Overall batch status reflects partial failure (not FAILED for entire batch) 4. Error message clearly identifies which table failed and why	**Critical**	Resilience
TC-NFR-005	**Concurrent export batches from multiple users processed correctly**	1. 3 different authenticated users available 2. STAGING environment	1. Simultaneously initiate export from User A, User B, and User C 2. All use different study_ids 3. Poll all three batch_ids to completion	1. All three batches return batch_id (202) without error 2. All three batches eventually reach COMPLETED 3. No cross-contamination of data between batches 4. Each user downloads only their own data	**High**	Non-Functional
TC-NFR-006	**Application fails fast with missing required environment config**	1. API server start script accessible 2. Required env var (e.g., DATABRICKS_HOST) removed from config	1. Remove a required environment variable 2. Attempt to start the API service 3. Capture startup output	1. Service fails to start 2. Error message clearly identifies the missing configuration variable by name 3. No partial initialization (other routes not accessible) 4. Exit code is non-zero	**High**	Resilience
TC-NFR-007	**Health check returns 503 when critical dependency is down**	1. Ability to temporarily disable Databricks SQL warehouse connection 2. /health endpoint deployed	1. Disable Databricks SQL warehouse connectivity 2. GET /health 3. Re-enable connectivity 4. GET /health again	1. First /health call returns HTTP 503 Service Unavailable 2. Response body shows Databricks component as unhealthy 3. Other healthy components still show healthy	**High**	Resilience

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
				4. After re-enable: /health returns 200 with all components healthy		
TC-NFR-008	**Retry logic with exponential backoff on transient failure**	1. Fault injection capability for transient Databricks timeout 2. Retry count and backoff configured	1. Configure first N attempts to fail with transient timeout 2. Initiate export 3. Monitor logs for retry attempts and timing	1. Retry attempts logged with attempt number 2. Delay between retries increases exponentially (e.g., 1s, 2s, 4s) 3. After configured max retries exhausted: table marked as FAILED 4. Transient error correctly identified and classified separately from permanent errors	**High**	Resilience

3.10 End-to-End — Complete User Journey

This scenario validates the complete end-to-end workflow from authentication through data download. It simulates the full journey of both an Enterprise user and a user, covering discovery, export initiation, status polling, and secure file retrieval in a single continuous flow.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-E2E-001	**Full user journey: authenticate → discover → export → poll → download**	1. test user with valid credentials for Study A 2. Study A enabled for Databricks Export API 3. Tables exist for Study A 4. Databricks Unity Catalog and Azure Blob Storage operational	1. Obtain OAuth2 bearer token using grant_type=dataexportapi for user 2. GET /api/export/tables to discover available tables; confirm tables for Study A present 3. POST /api/export with study_ids: [study_A_id], table_names from step 2, file_format: "PARQUET" 4. Record batch_id from 202 response 5. Poll GET /api/export/status/{batch_id} every 5 seconds until status=COMPLETED (max 3 minutes) 6. Extract download_url from COMPLETED response 7. HTTP GET to download_url; save ZIP file 8. Extract ZIP; inspect contents	1. Token obtained successfully 2. Tables listed include Study A tables (step 2: 200 OK, valid JSON) 3. Export initiated: 202 Accepted, valid batch_id returned (step 3) 4. Status transitions: INPROGRESS → COMPLETED 5. COMPLETED response contains: batch_id, status, table_count, total_rows, download_url 6. ZIP file downloaded successfully 7. ZIP contains one .parquet file per exported table 8. Files are valid PARQUET format; row counts match total_rows from status 9. End-to-end journey completes without errors	**Critical**	End-to-End
TC-E2E-002	**Full Enterprise user journey with CDF**	1. Enterprise test user with valid credentials for Customer A	1. Obtain OAuth2 bearer token for Enterprise user 2. POST /api/export with study_ids for all Customer A	1. Enterprise token accepted 2. CDF export initiated: 202, batch_id returned	**Critical**	End-to-End

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
	incremental export	2. Multiple studies under Customer A 3. CDF enabled; changes made in last 24 hours	studies, cdf_start_timestamp = 24 hours ago, file_format: "FEATHER" 3. Poll status until COMPLETED 4. Download and verify ZIP 5. Compare record count with expected changes in period	3. Export completes: COMPLETED with table_count and total_rows 4. ZIP downloaded successfully 5. ZIP contains .feather files 6. Record count in files matches expected incremental change count 7. Change metadata (_change_type, _commit_timestamp) present in each record		
TC-E2E-003	**Multi-environment config switch — same export works in DEV and STAGING**	1. DEV and STAGING environments configured 2. Test data seeded in both	1. Set ENV=DEV; start API; run POST /api/export; verify batch_id returned with DEV configuration 2. Check logs confirm environment = DEV 3. Set ENV=STAGING; restart API; run same export request 4. Check logs confirm environment = STAGING 5. Verify no code change required — only ENV var changed	1. Export works in both DEV and STAGING without code changes 2. Logs show correct environment name for each run 3. Health check response includes correct environment in both cases 4. Connection strings and credentials differ per environment automatically	**High**	End-to-End

3.11 Region Routing — Transparent Regional Request Forwarding

This scenario validates the region routing logic that transparently directs export requests to the appropriate regional API instance based on the study's associated region. Tests cover the full lookup chain (study → Instance_ID → Database_ID → Region_ID → regional endpoint), transparent redirection behavior, and failure modes at each routing step.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-RGN-001	**NA study request routed to NA regional instance**	1. Study in NA region; Instance_ID/Database_ID/Region_ID mapping configured 2. NA and EU regional API instances available (or mocked)	1. POST /api/export with study_id that belongs to NA region 2. Monitor routing logs	1. Request forwarded to NA regional API instance 2. Routing log confirms Region_ID = NA 3. Response (batch_id) returned transparently to caller 4. No region information leaked in response to user	**High**	Functional
TC-RGN-002	**EU study request routed to EU regional instance**	1. Study in EU region; lookup chain configured for EU	1. POST /api/export with study_id that belongs to EU region 2. Monitor routing logs	1. Request forwarded to EU_.DatabricksExport.API 2. Region_ID = EU confirmed in routing log 3. Response returned correctly to caller	**High**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-RGN-003	**Routing step 1 failure — study not found in registry**	1. Non-existent study_id prepared	1. POST /api/export with study_id not in registry 2. Include valid token	1. HTTP 404 returned 2. Error message: study not found 3. No routing attempted to regional instance 4. Failure logged	**High**	Functional
TC-RGN-004	**Regional instance unreachable — 503 with retry guidance**	1. Regional API instance for target region is offline/unreachable 2. Fault injection available	1. Take down target regional API instance 2. POST /api/export for study in that region	1. HTTP 503 Service Unavailable returned 2. Error message suggests retrying later 3. Routing failure logged 4. No batch_id returned	**High**	Resilience
TC-RGN-005	**Routing is transparent — caller receives normal response format**	1. Study correctly routed to regional instance 2. Regional instance operational	1. POST /api/export for study with known region 2. Inspect response structure	1. Response format identical to non-routed request: {batch_id, tables_queued} 2. No routing headers, region identifiers, or forwarding metadata in response 3. Polling and download URLs work correctly for routed batch	**High**	Functional

3.12 File Format Handling — PARQUET and FEATHER

This scenario validates the two supported file formats for exported data. Tests verify file extensions, ZIP compression types (ZIP_STORED for PARQUET, ZIP_DEFLATED for FEATHER), data type fidelity, and that format applies uniformly to all tables within a batch.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-FMT-001	**PARQUET export produces valid .parquet files in ZIP_STORED archive**	1. Authenticated user 2. file_format: "PARQUET" specified	1. POST /api/export with file_format: "PARQUET" 2. Poll until COMPLETED 3. Download ZIP; inspect with ZIP tool showing compression method 4. Extract .parquet file; open with Apache Parquet reader	1. ZIP archive created with ZIP_STORED (no compression) method 2. Files within ZIP have .parquet extension 3. Files are valid Apache Parquet columnar format 4. Data reads correctly without corruption 5. Column names and data types preserved correctly	**Critical**	Functional
TC-FMT-002	**FEATHER export produces valid .feather files in ZIP_DEFLATED archive**	1. Authenticated user 2. file_format: "FEATHER" specified	1. POST /api/export with file_format: "FEATHER" 2. Poll until COMPLETED 3. Download ZIP; inspect compression method 4. Extract .feather file; open with pyarrow.feather or R feather package	1. ZIP archive created with ZIP_DEFLATED compression 2. Files within ZIP have .feather extension 3. Files are valid Apache Feather format 4. Data reads correctly with pandas/R 5. Column types and values intact	**Critical**	Functional
TC-FMT-003	**Default format is PARQUET when**	1. Authenticated user 2. No file_format in request body	1. POST /api/export with body omitting file_format field 2. Poll	1. Export completes successfully 2. Files have .parquet extension 3. ZIP	**High**	Functional

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
	file_format omitted		until COMPLETED 3. Download and inspect file extensions	uses ZIP_STORED compression 4. Behavior identical to explicit file_format: "PARQUET"		
TC-FMT-004	**All tables in batch use the same file format**	1. Export batch with 3 tables, file_format: "FEATHER"	1. POST /api/export with 3 table_names and file_format: "FEATHER" 2. Download completed ZIP 3. Inspect all file extensions	1. All 3 files in ZIP have .feather extension 2. No mix of .parquet and .feather within same batch 3. Format consistency confirmed across all tables	**High**	Functional
TC-FMT-005	**Data type integrity preserved in PARQUET — strings, numerics, dates, nulls**	1. Test table with diverse column types: VARCHAR, INTEGER, DECIMAL, DATE, TIMESTAMP, NULL values	1. Export table with mixed data types in PARQUET format 2. Download and parse .parquet file 3. Compare values against source data in Databricks	1. VARCHAR columns have correct string values 2. INTEGER/DECIMAL columns have correct numeric values 3. DATE/TIMESTAMP columns have correct temporal values (correct timezone handling) 4. NULL values preserved as null (not empty string or 0) 5. No data type truncation or corruption	**High**	Functional
TC-FMT-006	**Invalid file_format value returns 400**	1. Authenticated user	1. POST /api/export with file_format: "XML" 2. POST /api/export with file_format: "csv" (lowercase)	1. First request: HTTP 400 Bad Request; error message lists valid options 2. Second request: HTTP 400 Bad Request (case-sensitive validation) 3. No exports initiated	**High**	Functional

3.13 Error Handling & Retry Logic

This scenario validates comprehensive error handling across the API. Tests cover structured error responses with error codes, table-level error details for partial failures, retry logic with exponential backoff, correct distinction between transient and permanent errors, and logging of all failures with sufficient context for troubleshooting.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-ERR-001	**Failed export returns detailed structured error response**	1. Export configured to fail 2. 2. Authenticated user	1. Initiate export that will fail (e.g., Databricks warehouse unavailable) 2. Poll status until final FAILED/ERROR state 3. Inspect error response	1. Status response contains: batch_id, status (FAILED or ERROR), message (descriptive) 2. Error indicates specific cause of failure 3. Error code present for programmatic handling 4. Timestamp present 5. No internal stack traces or DB queries exposed in user-facing message	**Critical**	Error Handling
TC-ERR-002	**Partial failure — table-level**	1. Export batch with 3 tables; 1 table will fail (access	1. Initiate 3-table export 2. Revoke access to one table after export starts	1. Overall status = ERROR (partial failure)	**Critical**	Error Handling

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
	errors included in response	revoked mid-export)	3. Poll until final state	2. Error response includes table-level detail: which table failed and why 3. Two successfully exported tables accessible 4. Failed table clearly identified in response		
TC-ERR-003	**Transient failure triggers automatic retry**	1. Fault injection configured to fail first 2 attempts with transient error 2. Retry count = 3	1. Initiate export 2. Configure first 2 table-level operations to fail with "connection timeout" 3. Poll to completion 4. Check logs	1. Export eventually succeeds (COMPLETED) after retries 2. Log entries show retry attempt 1 and 2 with failure reason 3. Log entry shows attempt 3 succeeded 4. Retry timestamps show exponential backoff interval	**High**	Resilience
TC-ERR-004	**Permanent failure after max retries exceeded**	1. All retry attempts configured to fail permanently 2. Retry count = 3	1. Configure all attempts for a table to fail 2. Initiate export 3. Monitor until final state	1. After 3 retries, table marked as permanently failed 2. Status = FAILED or ERROR 3. Log shows: "Max retries exceeded" with retry count 4. Error classified as permanent failure	**High**	Resilience
TC-ERR-005	**Permanent error NOT retried (e.g., invalid table permissions)**	1. Table with no Databricks read permission 2. Authenticated user with valid token	1. Request export for table without read access 2. Monitor logs during processing	1. Access denied error returned immediately 2. NO retry attempts logged (403 is permanent, not transient) 3. Table marked as failed with "permission denied" error 4. No exponential backoff applied	**High**	Error Handling
TC-ERR-006	**API requests logged with full context**	1. Logging infrastructure accessible 2. Valid and invalid requests ready	1. Make a successful POST /api/export 2. Make a request with invalid token 3. Access application logs	1. Successful request logged with: method=POST, path=/api/export, user_id, response_code=202 2. Failed auth logged with: method=POST, path=/api/export, response_code=401, reason 3. Correlation ID or request ID present for tracing 4. Token value NOT in logs	**High**	Error Handling

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-ERR-007	**Export lifecycle events logged**	1. Export initiated and completed	1. Initiate export 2. Monitor application logs throughout lifecycle	1. Log entry: export initiated (batch_id, user_id, table_count) 2. Log entry: export processing started per table 3. Log entry: export completed or failed per table 4. Log entry: final batch status (COMPLETED/FAILED) with summary	**High**	Error Handling
TC-ERR-008	**Log level configuration respected**	1. LOG_LEVEL environment variable configurable 2. Application restarted with different log levels	1. Set LOG_LEVEL=DEBUG; make requests; check logs 2. Set LOG_LEVEL=ERROR; make same requests; check logs	1. DEBUG level: verbose logs including debug-level messages present 2. ERROR level: only error entries present; debug/info messages absent 3. Log level switch requires only env var change, not code change	**Medium**	Error Handling

3.14 Health Check Endpoint — /health

This scenario validates the /health endpoint, which provides visibility into API and critical dependency health. It confirms correct HTTP status codes under healthy and degraded conditions, per-component health details, fast response times, and that authentication is not required.

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
TC-HLT-001	**Health check returns 200 when all systems healthy**	1. All dependencies (Databricks, Azure Blob, Database) operational 2. /health endpoint deployed	1. GET /health with no Authorization header 2. Parse response body	1. HTTP 200 OK 2. Response body contains overall status = "healthy" (or equivalent) 3. Response includes per-component status for: database, storage, Databricks 4. All components show healthy status 5. Response received in < 1000ms	**Critical**	Functional
TC-HLT-002	**Health check returns 503 when Databricks is down**	1. Databricks SQL warehouse connectivity disabled (fault injection)	1. Disable Databricks connectivity 2. GET /health	1. HTTP 503 Service Unavailable 2. Databricks component shows unhealthy in response body 3. Other healthy components (database,	**High**	Resilience

TC ID	Test Case Title	Precondition	Test Steps	Expected Result	Priority	Type
				storage) still show healthy 4. Overall status = unhealthy		
TC-HLT-003	**Health check does not require authentication**	1. No OAuth2 token available	1. GET /health without any Authorization header	1. HTTP 200 OK returned successfully 2. No 401 or 403 error 3. Full health status returned	**Critical**	Functional
TC-HLT-004	**Health check completes in < 1 second**	1. All dependencies healthy	1. GET /health 2. Measure time from request to response received 3. Repeat 5 times	1. All 5 responses received in < 1000ms 2. No individual call exceeds 1000ms 3. Average response time well below 1 second	**High**	Non-Functional
TC-HLT-005	**Health check includes environment name in response**	1. API deployed in STAGING environment (ENV=STAGING)	1. GET /health in STAGING 2. Inspect response body for environment indicator	1. Response body contains environment field with value "STAGING" (or equivalent) 2. Environment correctly reflects configured ENV variable 3. No hardcoded environment value	**Medium**	Functional

SECTION 4 — TEST COVERAGE SUMMARY

4.0 Overall Coverage Summary

Metric	Count
Total Test Cases	96
Critical Priority	36
High Priority	46
Medium Priority	14
Functional / E2E	64
Security	12
Non-Functional & Resilience	16
Endpoints Covered	4 (POST /api/export, GET /api/export/status/{batch_id}, GET /api/export/tables, GET /health)
Export Modes Covered	3 (Snapshot, CDF, Time-Travel)
File Formats Covered	2 (PARQUET, FEATHER)
User Types Covered	2 (Enterprise,)
Environments Covered	3 (Feature Branch, Dev Master and Stage5 — via TC-E2E-003)

4.1 Coverage by Scenario

Scenario	Critical	High	Medium	Low	Total
3.1 POST /api/export	**6**	**4**	2	0	**12**
3.2 GET /api/export/status/{batch_id}	**3**	**3**	2	0	**8**
3.3 GET /api/export/tables	**3**	**2**	1	0	**6**
3.4 Snapshot Export Mode	**2**	**2**	1	0	**5**
3.5 Change Data Feed (CDF) Export	**4**	**2**	1	0	**7**
3.6 Time-Travel Export Mode	**1**	**3**	1	0	**5**
3.7 Azure Blob Storage & SAS URL	**1**	**4**	1	0	**6**
3.8 Security & Data Isolation	**7**	**5**	0	0	**12**
3.9 Non-Functional & Resilience	**1**	**6**	1	0	**8**
3.10 End-to-End Journey	**2**	**1**	0	0	**3**
3.11 Region Routing	0	**4**	1	0	**5**
3.12 File Format Handling	**2**	**3**	1	0	**6**
3.13 Error Handling & Retry Logic	**2**	**5**	1	0	**8**
3.14 Health Check Endpoint	**2**	**2**	1	0	**5**
TOTAL	**36**	**46**	**14**	**0**	**96**

4.2 Coverage by Test Type

Test Type	Count	Coverage Notes
Functional	61	Happy paths + validation + edge cases for all endpoints and modes
End-to-End	3	Complete user journeys from auth to file download
Security	12	Auth vectors, authorization rules, cross-user isolation, IP allow-listing, SAS security
Non-Functional	9	Response time SLAs, parallel throughput, scalability under load
Error Handling / Resilience	7	Retry logic, structured errors, logging, health check degradation

4.3 EPIC Requirements Traceability Summary

All key EPIC requirements are traceable to at least one test case as follows:

EPIC Requirement Area	Covered By Scenarios
OAuth2 token validation, scope verification, custom grant_type	3.8 (TC-SEC-001 to 005)
Enterprise user validation, customer ID, IP allow-listing	3.8 (TC-SEC-007, 009, 010)
user validation, study access, study_id requirement	3.8 (TC-SEC-006, 008, 011)
Batch access authorization, ownership, 403 enforcement	3.8 (TC-SEC-006, 007)
Table discovery, metadata, permission filtering	3.3 (TC-TBL-001 to 006)
Gold schema tables, non-study lookup tables	3.3 (TC-TBL-004, 005)
Asynchronous export initiation, immediate batch_id return	3.1 (TC-EXP-001, 002), 3.9 (TC-NFR-001)
Snapshot export mode (default)	3.4 (TC-SNP-001 to 005)
CDF export mode, all DML ops, metadata	3.5 (TC-CDF-001 to 007)
Time-travel export, retention, version accuracy	3.6 (TC-TTR-001 to 005)
Mutual exclusivity of CDF and time-travel params	3.1 (TC-EXP-008), 3.6 (TC-TTR-003)
PARQUET and FEATHER file formats, compression types	3.12 (TC-FMT-001 to 006)
Parallel query execution, fault isolation	3.9 (TC-NFR-003, 004)
Azure Blob Storage write, naming, retention, SAS URLs	3.7 (TC-SDL-001 to 006)
Export request validation (400 errors)	3.1 (TC-EXP-008 to 012)
Error handling, partial failures, error codes	3.13 (TC-ERR-001 to 008)
Retry logic with exponential backoff	3.9 (TC-NFR-008), 3.13 (TC-ERR-003, 004, 005)
Structured logging, lifecycle events, sensitive data exclusion	3.13 (TC-ERR-006, 007, 008)
Health check endpoint, dependency checks, <1s response	3.14 (TC-HLT-001 to 005)
Region routing — Instance → Database → Region → forward	3.11 (TC-RGN-001 to 005)
Multi-environment configuration, startup validation	3.9 (TC-NFR-006), 3.10 (TC-E2E-003)
End-to-end user journey	3.10 (TC-E2E-001 to 003)

— End of QA Master Document —